

Web Technologies (9FC06) Viva-Voce Questions and Answers

Dr. Dheeraj Sundaragiri

June 5, 2025



Contents

1 Unit I: HTML 5 & CSS	2
1.1 Level 1 (Basic/Recall)	2
1.2 Level 2 (Understanding/Application)	2
1.3 Level 3 (Advanced/Analytical)	3
2 Unit II: JavaScript & Advanced JavaScript Concepts	5
2.1 Level 1 (Basic/Recall)	5
2.2 Level 2 (Understanding/Application)	5
2.3 Level 3 (Advanced/Analytical)	6
3 Unit III: Introduction to MERN & Node.js	8
3.1 Level 1 (Basic/Recall)	8
3.2 Level 2 (Understanding/Application)	8
3.3 Level 3 (Advanced/Analytical)	9
4 Unit IV: Web Servers & REST API	12
4.1 Level 1 (Basic/Recall)	12
4.2 Level 2 (Understanding/Application)	12
4.3 Level 3 (Advanced/Analytical)	13
5 Unit V: Express Framework & MongoDB	16
5.1 Level 1 (Basic/Recall)	16
5.2 Level 2 (Understanding/Application)	16
5.3 Level 3 (Advanced/Analytical)	17
6 Unit VI: Introduction to ReactJS	20
6.1 Level 1 (Basic/Recall)	20
6.2 Level 2 (Understanding/Application)	20
6.3 Level 3 (Advanced/Analytical)	22

1 Unit I: HTML 5 & CSS 3

1.1 Level 1 (Basic/Recall)

1. **Q:** What does HTML stand for?
A: HyperText Markup Language.
2. **Q:** Name three new semantic elements in HTML5.
A: Some examples include `<article>`, `<aside>`, `<nav>`, `<section>`, `<header>`, `<footer>`. (Accept any three valid ones).
3. **Q:** What is the purpose of the `<header>` element?
A: It represents introductory content, typically a group of introductory or navigational aids. It may contain heading elements, a logo, a search form, etc.
4. **Q:** What is Web Storage API? Name its two types.
A: Web Storage API provides mechanisms for web applications to store data locally within the user's browser. Its two types are `localStorage` and `sessionStorage`.
5. **Q:** What does a 404 HTTP status code mean?
A: It means "Not Found" – the server cannot find the requested resource.
6. **Q:** What is CSS?
A: CSS stands for Cascading Style Sheets. It's a style sheet language used for describing the presentation of a document written in a markup language like HTML.
7. **Q:** What are the three types of CSS?
A: Inline, Internal (or Embedded), and External CSS.
8. **Q:** Explain the CSS box model components.
A: The CSS box model describes the rectangular boxes that are generated for elements in the document tree. Its components are: content, padding, border, and margin.
9. **Q:** What is a media query?
A: A media query is a CSS3 technique that allows content rendering to adapt to conditions such as screen resolution (e.g., smartphone screen vs. desktop screen).
10. **Q:** What is Tailwind CSS?
A: Tailwind CSS is a utility-first CSS framework for rapidly building custom user interfaces.

1.2 Level 2 (Understanding/Application)

11. **Q:** How does a semantic element like `<article>` differ from a `<div>`?
A: A `<article>` element semantically represents a self-contained composition in a document (e.g., a blog post, a news story), conveying meaning about its content. A `<div>` is a generic container with no semantic meaning, used for grouping or styling.
12. **Q:** When would you use `localStorage` versus `sessionStorage`?
A: Use `localStorage` for persistent data that should remain available even after the browser window is closed and reopened. Use `sessionStorage` for data that should only last for the duration of the page session (until the browser tab is closed).
13. **Q:** Explain the difference between a 200 OK and a 301 Moved Permanently HTTP status code.
A: A 200 OK status code indicates that the request has succeeded. A 301 Moved Permanently status code indicates that the requested resource has been assigned a new permanent URI and any future references to this resource should use one of the returned URIs.
14. **Q:** Describe the syntax structure of a CSS rule.
A: A CSS rule consists of a selector and a declaration block. The selector points to the HTML element to be styled. The declaration block contains one or more declarations separated by semicolons, where each declaration includes a CSS property name and a value, separated by a colon. Example: `p { color: blue; }`.

15. **Q:** How does **inline** CSS differ from **external** CSS in terms of application and maintenance?
A: Inline CSS is applied directly to a single HTML element using the **style** attribute, making it highly specific but difficult to maintain for larger sites. External CSS is written in separate **.css** files, linked to HTML documents, allowing styles to be shared across multiple pages and making maintenance easier.
16. **Q:** Explain the difference between **margin** and **padding** in the CSS box model.
A: **Padding** is the space between the content of an element and its border. **Margin** is the space outside the border, between the element and other elements.
17. **Q:** How does CSS Grid differ from CSS Flexbox in layout capabilities?
A: CSS Flexbox is primarily designed for one-dimensional layouts (either a row or a column). CSS Grid is designed for two-dimensional layouts (rows and columns simultaneously), making it more suitable for complex page structures.
18. **Q:** What is the role of the viewport meta tag in responsive web design?
A: The viewport meta tag (`<meta name="viewport" content="width=device-width, initial-scale=1.0">`) controls the layout on mobile browsers. It sets the width of the viewport to the device width and the initial zoom level.
19. **Q:** How can you create a simple fade-in transition using CSS?
A: You can use the **transition** property along with **opacity**. For example, an element might initially have **opacity: 0;** and on hover or when a class is added, its **opacity** changes to **1;**. The **transition: opacity 0.5s ease-in-out;** property would make this change gradual.
20. **Q:** What are the advantages of using a CSS framework like Tailwind CSS?
A: Advantages include rapid development due to utility classes, highly customizable designs without writing custom CSS, consistency in styling, and often better performance due to purgeable unused styles.

1.3 Level 3 (Advanced/Analytical)

21. **Q:** Discuss the accessibility benefits of using HTML5 semantic elements.
A: Semantic elements like `<nav>`, `<main>`, `<article>`, and `<aside>` provide a clear structure to web content. This helps assistive technologies (like screen readers) understand the purpose of different sections of a page, making it easier for users with disabilities to navigate and comprehend the content.
22. **Q:** What are the security considerations when using Web Storage API?
A: Data stored in Web Storage is not automatically encrypted and is susceptible to Cross-Site Scripting (XSS) attacks. If an attacker injects script into a page, they can access all data stored in **localStorage** or **sessionStorage** for that domain. Therefore, sensitive information (like tokens or PII) should be handled with extreme care or avoided in Web Storage.
23. **Q:** Imagine a website is returning a 503 Service Unavailable error. What could be the potential causes?
A: Potential causes include: the server is overloaded with requests and cannot handle more; the server is down for maintenance; there's an issue with a backend service the server relies on; a firewall or load balancer is misconfigured or has reached its limit.
24. **Q:** How does CSS specificity work, and why is it important?
A: Specificity determines which CSS rule applies if multiple rules target the same element and property. Rules with higher specificity override rules with lower specificity. It's calculated based on the number of ID selectors, class/attribute/pseudo-class selectors, and type/pseudo-element selectors. Understanding specificity is crucial for debugging CSS and ensuring styles apply as expected. Inline styles have the highest specificity.
25. **Q:** Explain how Flexbox **justify-content** and **align-items** properties work together.
A: **justify-content** aligns flex items along the main axis (e.g., horizontally if **flex-direction** is row). **align-items** aligns flex items along the cross axis (e.g., vertically if **flex-direction** is row). They work together to position items within the flex container in two dimensions.

26. **Q:** Describe a scenario where using CSS Grid would be more advantageous than Flexbox for a complex layout.
A: For an overall page layout with distinct regions like a header, sidebar, main content area, and footer, CSS Grid is more advantageous. It allows you to define rows and columns for the entire page structure and place items precisely within this grid, which is harder and less intuitive with Flexbox alone.
27. **Q:** How would you debug responsive design issues on different devices?
A: Use browser developer tools (like Chrome DevTools or Firefox Developer Tools) which offer device emulation modes. Test on actual physical devices. Use remote debugging tools. Check media queries, viewport settings, and CSS rules for potential conflicts or errors. Simplify the CSS to isolate the problem.
28. **Q:** What is the difference between a CSS transition and a CSS animation? When would you use each?
A: CSS transitions are used to animate an element from a start state to an end state, typically triggered by a change in state (e.g., hover, focus, class change). CSS animations allow for more complex, multi-step animations with keyframes, offering finer control over the animation sequence, duration, and iteration. Use transitions for simple state changes and animations for more elaborate visual effects.
29. **Q:** How does Tailwind's utility-first approach compare to traditional BEM or SMACSS methodologies?
A: Tailwind provides low-level utility classes directly in HTML, leading to rapid prototyping without writing custom CSS. BEM (Block, Element, Modifier) and SMACSS (Scalable and Modular Architecture for CSS) focus on creating reusable, semantic CSS components with specific naming conventions to manage complexity in larger projects. Tailwind can feel less "semantic" in HTML but offers high customizability and avoids CSS bloat if configured with PurgeCSS.
30. **Q:** If you needed to support very old browsers that don't understand HTML5 semantic tags or CSS3 features, what strategies would you employ?
A: Use JavaScript shims/polyfills (like HTML5Shiv) to enable styling of HTML5 elements in older browsers. Provide fallback CSS for CSS3 features or use vendor prefixes for broader compatibility if needed. Employ progressive enhancement: build a basic functional experience for all browsers and add advanced features for modern browsers. Test thoroughly.

2 Unit II: JavaScript & Advanced JavaScript Concepts

2.1 Level 1 (Basic/Recall)

1. **Q:** What is JavaScript?
A: JavaScript is a high-level, interpreted programming language primarily known as the scripting language for Web pages, but it's also used in many non-browser environments thanks to Node.js.
2. **Q:** Name three primitive data types in JavaScript.
A: String, Number, Boolean, Null, Undefined, Symbol, BigInt. (Accept any three).
3. **Q:** What is a JavaScript function?
A: A JavaScript function is a block of code designed to perform a particular task. It is executed when "something" invokes it (calls it).
4. **Q:** How do you declare an array in JavaScript?
A: Using array literal syntax: `let myArray = [1, 2, 3];` or using the Array constructor: `let myArray = new Array(1, 2, 3);`.
5. **Q:** What is an object in JavaScript?
A: An object in JavaScript is a standalone entity, with properties and type. It's often used to store collections of data and more complex entities.
6. **Q:** What does the `let` keyword do?
A: The `let` keyword declares a block-scoped local variable, optionally initializing it to a value.
7. **Q:** What is an arrow function?
A: An arrow function (`=>`) is a concise way to write function expressions in JavaScript. They have a shorter syntax and lexically bind the `this` value.
8. **Q:** What is a callback function?
A: A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.
9. **Q:** What is a Promise?
A: A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value.
10. **Q:** What does `async/await` do?
A: `async/await` is syntactic sugar built on top of Promises, allowing asynchronous, promise-based code to be written in a cleaner style that looks more like synchronous code, avoiding promise chains.

2.2 Level 2 (Understanding/Application)

11. **Q:** Explain the difference between `null` and `undefined` in JavaScript.
A: `undefined` means a variable has been declared but has not yet been assigned a value. `null` is an assignment value; it can be assigned to a variable as a representation of no value or an empty object.
12. **Q:** How do you pass parameters to a function and return a value?
A: Parameters are listed inside the parentheses in the function definition. Arguments are the values received by the function when it is invoked. A value is returned from a function using the `return` statement. Example: `function add(a, b) { return a + b; }.`
13. **Q:** Provide an example of accessing an element in an array and a property in an object.
A: Array: `let fruits = ["apple", "banana"]; console.log(fruits[0]); // apple.` Object: `let person = { name: "John", age: 30 }; console.log(person.name); // John` or `console.log(person["age"]); // 30.`
14. **Q:** What is the purpose of regular expressions? Give a simple example.
A: Regular expressions are patterns used to match character combinations in strings. They are used for searching, replacing, and validating text. Example: `/d+/g` matches one or more digits globally in a string.

15. **Q:** Explain the difference between `let`, `const`, and `var` in terms of scope and reassignment.
A: `var` is function-scoped or globally-scoped, and can be redeclared and reassigned. `let` is block-scoped (`{}`), cannot be redeclared in the same scope, but can be reassigned. `const` is also block-scoped, cannot be redeclared, and cannot be reassigned (though for objects/arrays, their properties/elements can be changed).
16. **Q:** How does destructuring assignment work for arrays and objects? Provide an example for each.
A: Destructuring assignment is a syntax that allows unpacking values from arrays, or properties from objects, into distinct variables. Array: `let [a, b] = [1, 2]; // a=1, b=2`. Object: `let {name, age} = {name: "Alice", age: 25}; // name="Alice", age=25`.
17. **Q:** What is the difference between the spread (`...`) and rest (`...`) operators?
A: The spread operator expands an iterable (like an array or string) into individual elements, or an object into key-value pairs. It's used in places like function calls or array/object literals. The rest parameter syntax collects multiple function arguments into a single array parameter. The syntax is the same (`...`), but the usage context determines its meaning.
18. **Q:** Explain prototypal inheritance in JavaScript with a simple example.
A: Prototypal inheritance is a feature in JavaScript where objects can inherit properties and methods from other objects (their prototype). When trying to access a property of an object, JavaScript will first look at the object itself, and if not found, it will look at its prototype, then the prototype's prototype, and so on, up the prototype chain. Example: `let animal = { eat: function() { console.log("eating..."); } }; let dog = Object.create(animal); dog.bark = function() { console.log("woof!"); }; dog.eat(); // "eating..."`
19. **Q:** What is a closure? Provide a use case.
A: A closure is a function that has access to its own scope, the scope of the outer function, and the global scope, even after the outer function has finished executing. Use case: Creating private variables/methods, data encapsulation, or in event handlers where the handler needs access to variables from its surrounding scope.
20. **Q:** How does a Promise help in handling asynchronous operations compared to traditional callbacks?
A: Promises provide a cleaner and more manageable way to handle asynchronous operations by avoiding "callback hell" (deeply nested callbacks). They allow for better error handling through `.catch()` and chaining of asynchronous operations using `.then()`.

2.3 Level 3 (Advanced/Analytical)

21. **Q:** Discuss event bubbling and capturing in JavaScript.
A: Event bubbling and capturing are two phases of event propagation in the HTML DOM. Capturing phase: The event travels down from the window to the target element. Bubbling phase: The event travels up from the target element to the window. By default, event handlers are registered in the bubbling phase. One can use `addEventListener(event, handler, useCapture)` to specify the phase.
22. **Q:** How does the `this` keyword work in JavaScript, and how can its context change?
A: The value of `this` depends on how a function is called (its invocation context). In the global scope, `this` refers to the global object (window in browsers). In an object method, `this` refers to the object. In a simple function call (strict mode: undefined, non-strict: global object), its value can be tricky. Arrow functions lexically bind `this` from their surrounding scope. Methods like `call()`, `apply()`, and `bind()` can explicitly set the `this` context.
23. **Q:** Explain the concept of hoisting in JavaScript.
A: Hoisting is JavaScript's default behavior of moving declarations to the top of the current scope (script or function) before code execution. Variable declarations using `var` are hoisted (and initialized with `undefined`), and function declarations are also hoisted (and can be called before they appear in code). Declarations with `let` and `const` are hoisted but not initialized, leading to a "temporal dead zone" until their declaration is encountered.

24. **Q:** How would you create a deep copy of an object in JavaScript?
A: A shallow copy shares references to nested objects/arrays. For a deep copy (all nested objects/arrays are also copied): 1. Using `JSON.parse(JSON.stringify(obj))`: Simple but has limitations (e.g., loses functions, Date objects become strings, undefined properties are removed). 2. Using a recursive function that iterates through properties and copies them, creating new objects/arrays for nested structures. 3. Using external libraries like Lodash's `_cloneDeep()`.
25. **Q:** What are common use cases for the `map()`, `filter()`, and `reduce()` array methods?
A: `map()`: Creates a new array by applying a function to each element of the original array (e.g., transforming a list of numbers into their squares). `filter()`: Creates a new array with all elements that pass a test implemented by the provided function (e.g., filtering out even numbers from a list). `reduce()`: Executes a reducer function on each element of the array, resulting in a single output value (e.g., summing all numbers in an array, or flattening an array of arrays).
26. **Q:** Describe a scenario where using an arrow function might behave differently than a traditional function expression, especially concerning `this`.
A: Arrow functions do not have their own `this` binding. They inherit `this` from the enclosing lexical scope. This is useful in callbacks within methods. For example, in an object method, if you use a traditional function for a `setTimeout` callback, `this` inside the callback will not refer to the object. If you use an arrow function, `this` will correctly refer to the object. Conversely, arrow functions are not suitable as object methods if you need `this` to refer to the object itself.
27. **Q:** How does prototypal inheritance differ from classical inheritance found in languages like Java or C++?
A: Classical inheritance involves classes acting as blueprints for objects, with inheritance occurring between classes. Prototypal inheritance involves objects inheriting directly from other objects. JavaScript uses prototypes; while ES6 classes provide syntactic sugar, they still work on top of the prototypal model. There are no true "classes" in the classical sense, only objects and their prototype links.
28. **Q:** Explain the event loop in JavaScript and how it handles asynchronous code.
A: JavaScript is single-threaded, meaning it can do only one thing at a time. The event loop is a mechanism that allows Node.js or browsers to perform non-blocking I/O operations despite this. When an asynchronous operation (like a timer, network request, or file I/O) is initiated, it's offloaded to system APIs. Once the operation completes, its callback function is placed in a callback queue (or task queue). The event loop continuously checks if the call stack is empty. If it is, it takes the first message from the queue and pushes its callback onto the call stack for execution.
29. **Q:** How does `async/await` improve upon Promises in terms of code readability and error handling?
A: `async/await` allows writing asynchronous code that looks and behaves a bit more like synchronous code, making it easier to read and understand complex asynchronous flows by avoiding nested `.then()` chains. Error handling can be done using standard `try...catch` blocks, which is often more intuitive than `.catch()` methods in Promise chains, especially for multiple async operations.
30. **Q:** Discuss potential pitfalls of closures, such as memory leaks.
A: Closures can lead to memory leaks if variables in the outer scope are no longer needed but are kept alive because the inner function still holds a reference to them. This is particularly common with event listeners or callbacks that are not properly removed, as they retain their lexical environment. Another pitfall can be creating closures in loops incorrectly, where all iterations might inadvertently share the same lexical environment for a variable.

3 Unit III: Introduction to MERN & Node.js

3.1 Level 1 (Basic/Recall)

1. **Q:** What does MERN stand for?
A: MongoDB, Express.js, React, Node.js.
2. **Q:** Name the four components of the MERN stack.
A: MongoDB (database), Express.js (backend web framework), React (frontend library), and Node.js (JavaScript runtime environment).
3. **Q:** What is Node.js?
A: Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that executes JavaScript code outside a web browser.
4. **Q:** What is REPL in Node.js?
A: REPL stands for Read-Eval-Print Loop. It's an interactive command-line interface where you can enter Node.js commands, have them evaluated, and see the results printed.
5. **Q:** What is a Node module?
A: A Node module is a reusable block of code, organized in one or more JavaScript files, that encapsulates related functionality. Modules can be built-in, third-party, or custom.
6. **Q:** Name one built-in Node.js module.
A: Examples include **fs** (File System), **http** (HTTP networking), **os** (Operating System information), **path** (File paths). (Accept any one).
7. **Q:** What is the purpose of the **http** module in Node.js?
A: The **http** module is used to create HTTP servers and clients, allowing Node.js applications to transfer data over HyperText Transfer Protocol.
8. **Q:** What are environment variables?
A: Environment variables are key-value pairs that are external to an application and can configure its behavior in different environments (e.g., development, production) without changing the code.
9. **Q:** What is the **dotenv** package used for?
A: The **dotenv** package is used to load environment variables from a **.env** file into **process.env** in a Node.js application, making it easier to manage configuration settings.
10. **Q:** What is server-less computing in simple terms?
A: Server-less computing is a cloud execution model where the cloud provider dynamically manages the allocation and provisioning of servers. Developers write and deploy code in functions, and pay only for the compute time consumed.

3.2 Level 2 (Understanding/Application)

11. **Q:** Describe the role of each component (MongoDB, Express.js, React, Node.js) in the MERN stack.
A: **MongoDB:** A NoSQL database used to store application data in flexible, JSON-like documents. **Express.js:** A back-end web application framework running on Node.js, used to build APIs and handle HTTP requests/responses. **React:** A front-end JavaScript library for building user interfaces and interactive UI components. **Node.js:** A JavaScript runtime environment that allows JavaScript to be run on the server-side, powering the backend (Express.js runs on Node.js).
12. **Q:** Why might someone choose the MERN stack for a web application?
A: Key reasons include using JavaScript across the entire stack (reducing context switching), its suitability for building fast and scalable applications, a large and active community, and the flexibility of MongoDB's schema-less nature.
13. **Q:** How does Node.js handle asynchronous operations?
A: Node.js uses an event-driven, non-blocking I/O model. It employs an event loop and worker threads (for some operations) to handle asynchronous tasks like file system operations, network requests, or database queries without blocking the main thread. Callbacks, Promises, and **async/await** are used to manage asynchronous code flow.

14. **Q:** Explain the purpose of `module.exports` and `require()` in Node.js.
A: `module.exports` is an object that is used to export functionalities (like functions, objects, or variables) from a Node.js module so they can be used in other modules. `require()` is a built-in function used to import these exported functionalities from other modules into the current module.
15. **Q:** How can you read a file using the Node.js `fs` module? (Describe the general process)
A: You use methods like `fs.readFile()` (asynchronous) or `fs.readFileSync()` (synchronous). For the asynchronous version, you provide the file path, encoding (optional), and a callback function that receives an error object (if any) and the file content as arguments.
16. **Q:** How can you access environment variables in a Node.js application?
A: Environment variables are accessible through the `process.env` object. For example, if you have an environment variable named `API_KEY`, you can access it in Node.js as `process.env.API_KEY`.
17. **Q:** What is the benefit of using `dotenv` to manage environment variables?
A: `dotenv` allows developers to store environment-specific configurations (like API keys, database credentials) in a `.env` file in the project root, which is typically gitignored. This keeps sensitive information out of the codebase and makes it easy to manage different configurations for development, testing, and production environments.
18. **Q:** Explain the concept of the event loop in Node.js.
A: The event loop allows Node.js to perform non-blocking I/O operations. It continuously checks the call stack to see if it's empty. If it is, it looks at the event queue (or callback queue) for pending asynchronous operations. If there are any, it moves the first callback to the call stack for execution. This cycle allows a single thread to handle many concurrent operations.
19. **Q:** What kind of applications is Node.js well-suited for?
A: Node.js is well-suited for I/O-bound applications, such as real-time applications (chat, gaming servers), data streaming applications, APIs (REST, GraphQL), microservices, and single-page applications that require a backend. It's generally not ideal for CPU-intensive tasks without offloading them.
20. **Q:** How can the `os` module be useful in a Node.js application?
A: The `os` module provides operating system-related utility methods and properties. It can be used to get information like CPU architecture (`os.arch()`), free memory (`os.freemem()`), total memory (`os.totalmem()`), network interfaces (`os.networkInterfaces()`), platform (`os.platform()`), and EOL (end-of-line) marker. This can be useful for logging, monitoring, or platform-specific logic.

3.3 Level 3 (Advanced/Analytical)

21. **Q:** Compare and contrast server-based and server-less architectures. What are the pros and cons of server-less for a "Hello World" application?
A: **Server-based:** You provision and manage servers (physical or virtual). Pros: Full control, predictable costs for consistent load. Cons: Management overhead, can be over-provisioned (costly) or under-provisioned (performance issues). **Server-less:** Cloud provider manages servers. You deploy functions. Pros: No server management, auto-scaling, pay-per-use. Cons: Potential for cold starts, vendor lock-in, statelessness limitations. For "Hello World": Server-less is very simple to deploy and cost-effective (likely free tier), but might have a slight delay on the first request (cold start). Server-based is overkill and more setup.
22. **Q:** Discuss the single-threaded nature of Node.js. How does it achieve concurrency?
A: Node.js runs JavaScript code in a single main thread. It achieves concurrency not through multi-threading for user code execution, but through an event-driven architecture and non-blocking I/O operations. Asynchronous tasks are offloaded to the system kernel or a worker thread pool. When these tasks complete, their callbacks are placed in an event queue and processed by the event loop when the main thread is free. This allows Node.js to handle many connections efficiently without the overhead of managing many threads.
23. **Q:** What are streams in Node.js, and why are they useful? Provide an example use case.
A: Streams are instances of `EventEmitter` that handle reading or writing data in a continuous flow, piece by piece (chunks), rather than loading entire data into memory. They are useful for working

with large amounts of data, such as reading/writing large files, handling network communication, or processing data pipelines. Example use case: Piping a readable stream from a large file to a writable stream representing an HTTP response to send the file to a client without loading the entire file into memory. `fs.createReadStream('large.mp4').pipe(res);`

24. **Q:** Explain the difference between `Buffer` and `string` in Node.js, especially concerning file I/O.

A: A `string` in JavaScript is a sequence of characters (typically UTF-16). A `Buffer` in Node.js is used to handle binary data. It represents a fixed-length sequence of bytes. When reading files, especially non-text files (images, videos) or when specific encodings are involved, data is often handled as `Buffers`. You can convert `Buffers` to strings with specific encodings (`buf.toString('utf8')`) and vice-versa. Using `Buffers` is more efficient for binary data manipulation.

25. **Q:** How would you handle errors when working with asynchronous operations like file I/O in Node.js?

A: 1. **Callbacks:** The "error-first callback" pattern is common, where the first argument to the callback function is an error object (or null if no error). Always check this first argument.

2. **Promises:** Use `.then(successCallback, errorCallback)` or `.then(successCallback).catch(errorCallback)` to handle resolved values or rejections. 3. **Async/Await:** Use `try...catch` blocks around `await` calls to handle rejected promises as synchronous-like errors.

26. **Q:** Why is it generally recommended not to block the main thread in Node.js? What are the consequences?

A: Blocking the main thread means executing a long-running synchronous operation (like complex calculations or synchronous I/O). Since Node.js uses a single main thread for executing JavaScript and handling events, blocking it prevents the event loop from processing any other incoming requests or pending operations. Consequences include poor performance, unresponsiveness, and an inability to handle concurrent users effectively.

27. **Q:** Discuss different strategies for managing Node.js application configuration beyond just `dotenv` for different environments (development, staging, production).

A: 1. **Environment-specific .env files:** E.g., `.env.development`, `.env.production`, loaded based on `NODE_ENV`. 2. **Configuration files:** Using JSON or JavaScript files (e.g., `config/default.json`, `config/production.json`) and libraries like `config` or `nconf` that allow hierarchical configurations and overrides. 3. **Cloud provider configuration services:** Services like AWS Parameter Store, Azure App Configuration, or Google Cloud Secret Manager for securely storing and managing configuration, especially secrets. 4. **Command-line arguments:** Passing configuration via CLI flags.

28. **Q:** How does the Node.js `events` module facilitate an event-driven architecture?

A: The `events` module provides the `EventEmitter` class. Objects that are `EventEmitter` instances can emit named events that cause registered listener functions to be called. This allows different parts of an application to react to occurrences (events) without being tightly coupled. Many Node.js built-in modules (like HTTP servers, streams) inherit from `EventEmitter`. It's fundamental to Node.js's non-blocking, asynchronous nature.

29. **Q:** What are some security best practices when working with environment variables in a Node.js application?

A: 1. Never commit `.env` files containing sensitive information to version control (add `.env` to `.gitignore`). 2. Use a template file (e.g., `.env.example`) to show required variables. 3. Restrict access to environment variables on servers (e.g., proper file permissions, secure storage for production secrets). 4. Avoid logging sensitive environment variables. 5. Regularly audit and rotate secrets stored in environment variables. 6. Use dedicated secret management systems for production environments.

30. **Q:** If you need to perform CPU-intensive tasks in a Node.js application, what approaches can you take to avoid blocking the event loop?

A: 1. **Worker Threads:** Use the `worker_threads` module to run JavaScript code in parallel threads, offloading CPU-bound tasks from the main thread. 2. **Child Processes:** Use the `child_process` module (e.g., `fork()`) to spawn separate processes for heavy computations. 3. **Task Queues:** Delegate tasks to a separate microservice or a task queue system (like RabbitMQ,

Kafka, Redis queues) that can be processed by dedicated workers (possibly written in other languages better suited for CPU tasks). 4. **Native Addons:** Write CPU-intensive parts in C++ and expose them as Node.js addons for better performance. 5. **Break down tasks:** If possible, break down long tasks into smaller chunks and process them asynchronously using `setImmediate()` or `process.nextTick()` to yield control to the event loop periodically.

4 Unit IV: Web Servers & REST API

4.1 Level 1 (Basic/Recall)

1. **Q:** What is a web server?
A: A web server is software and underlying hardware that accepts requests via HTTP (or HTTPS) from clients (typically web browsers) and responds with content, such as HTML pages, images, or data.
2. **Q:** Describe the client-server architecture in one sentence.
A: Client-server architecture is a distributed application structure that partitions tasks or workloads between providers of a resource or service, called servers, and service requesters, called clients.
3. **Q:** What is an HTTP request?
A: An HTTP request is a message sent by a client (e.g., a browser) to a server to ask for a specific resource or action.
4. **Q:** What is an HTTP response?
A: An HTTP response is a message sent by a server back to a client, containing the requested resource or a status of the request.
5. **Q:** What is a REST API?
A: A REST (Representational State Transfer) API is an architectural style for designing networked applications, using standard HTTP methods to operate on resources identified by URIs.
6. **Q:** Name two common HTTP verbs used in REST APIs.
A: GET, POST. (Others include PUT, DELETE, PATCH).
7. **Q:** What does the **GET** verb typically do in a REST API?
A: The **GET** verb is typically used to retrieve or read a representation of a resource.
8. **Q:** What does the **POST** verb typically do in a REST API?
A: The **POST** verb is typically used to create a new resource or submit data to be processed by a resource.
9. **Q:** What is a resource in the context of REST APIs?
A: A resource is any information or entity that can be named and accessed, such as a user, a product, or an order. It's identified by a URI.
10. **Q:** What is an API endpoint?
A: An API endpoint is a specific URL where an API can be accessed by a client to perform an operation on a resource.

4.2 Level 2 (Understanding/Application)

11. **Q:** Explain the typical flow of a request-response cycle in a client-server architecture.
A: 1. Client initiates a connection and sends an HTTP request to the server (e.g., for a webpage). 2. The server receives and processes the request. 3. The server may interact with a database or other services. 4. The server formulates an HTTP response, including a status code and often a body (e.g., HTML, JSON). 5. The server sends the response back to the client. 6. The client receives the response and processes it (e.g., renders the webpage).
12. **Q:** What are some common components of an HTTP request message?
A: Common components include: 1. **Request Line:** HTTP method (GET, POST), Request URI, HTTP version. 2. **Headers:** Key-value pairs providing information about the request (e.g., **Host**, **User-Agent**, **Accept**, **Content-Type**). 3. **Message Body (optional):** Contains data sent with the request (e.g., form data in a POST request).
13. **Q:** What are some common components of an HTTP response message (e.g., status line, headers, body)?
A: Common components include: 1. **Status Line:** HTTP version, Status Code (e.g., 200), Reason Phrase (e.g., OK). 2. **Headers:** Key-value pairs providing information about the response (e.g., **Content-Type**, **Content-Length**, **Date**). 3. **Message Body (optional):** Contains the requested resource or data (e.g., HTML, JSON).

14. **Q:** How can you create a very basic HTTP server using Node.js (conceptually)?
A: You would use the built-in `http` module. The process involves: 1. Requiring the `http` module. 2. Calling `http.createServer()` with a callback function that takes `request` and `response` objects as arguments. 3. Inside the callback, you'd inspect the request (e.g., URL, method) and use the response object to set headers (e.g., `res.writeHead()`) and send back data (`res.end()`). 4. Calling the `listen()` method on the created server object, specifying a port number.
15. **Q:** What are the key principles of REST (Representational State Transfer)?
A: Key principles include: 1. **Client-Server Architecture:** Separation of concerns. 2. **Statelessness:** Each request from client to server must contain all information needed to understand the request. The server does not store client context between requests. 3. **Cacheability:** Responses must define themselves as cacheable or not. 4. **Uniform Interface:** A consistent way of interacting with resources (using URIs, HTTP verbs, standard media types). 5. **Layered System:** Client cannot ordinarily tell whether it is connected directly to the end server or to an intermediary. 6. **Code on Demand (optional):** Servers can temporarily extend client functionality by transferring executable code (e.g., JavaScript).
16. **Q:** Explain the purpose of the PUT verb and how it differs from POST.
A: The PUT verb is typically used to update an existing resource completely, or create a resource if it doesn't exist at a known URI. The client specifies the full URI of the resource. POST is generally used to create a new resource under a parent resource (server often determines the URI of the new resource) or for operations that don't fit other verbs. PUT is idempotent (multiple identical requests have the same effect as one), while POST is generally not.
17. **Q:** Explain the purpose of the DELETE verb.
A: The DELETE verb is used to remove or delete a specific resource identified by a URI.
18. **Q:** What does "statelessness" mean in the context of REST APIs?
A: Statelessness means that the server does not store any information about the client's state between requests. Each request from the client must contain all the information necessary for the server to understand and process it (e.g., authentication tokens, identifiers). This simplifies server design and improves scalability.
19. **Q:** How are parameters typically passed in a GET request?
A: Parameters in a GET request are typically appended to the URL as a query string. The query string starts with a `?` and consists of key-value pairs separated by `&`. Example: `/users?id=123&status=active`.
20. **Q:** Why is JSON a popular data format for REST APIs?
A: JSON (JavaScript Object Notation) is popular because it's lightweight, human-readable, easy for machines to parse and generate, and maps well to data structures in most programming languages (especially JavaScript). It has broad support across platforms and languages.

4.3 Level 3 (Advanced/Analytical)

21. **Q:** Discuss the difference between a web server (like Apache or Nginx) and an application server.
A: A **web server** primarily serves static content (HTML, CSS, images) and handles HTTP requests. Examples: Apache HTTP Server, Nginx. An **application server** provides the business logic for an application. It can serve dynamic content, manage transactions, handle security, and interact with databases. It often sits behind a web server or can include web server capabilities. Node.js with Express can act as an application server. Often, they are used together: Nginx might act as a reverse proxy and serve static files, forwarding dynamic requests to an application server like one built with Node.js/Express.
22. **Q:** How do HTTP headers play a role in communication between client and server (e.g., **Content-Type**, **Authorization**)?
A: HTTP headers provide metadata about the request or response. **Content-Type:** Indicates the media type of the message body (e.g., `application/json`, `text/html`). **Authorization:** Carries credentials for authenticating the client with the server (e.g., Bearer token). Other headers control caching (**Cache-Control**), content negotiation (**Accept**), connection management (**Connection**), and more, enabling rich communication semantics.

23. **Q:** What are some common HTTP status codes a REST API might return and what do they indicate (e.g., 201, 400, 401, 403, 500)?
A: **201 Created:** The request has been fulfilled and resulted in a new resource being created. **400 Bad Request:** The server cannot or will not process the request due to something perceived to be a client error (e.g., malformed request syntax, invalid request message framing, or deceptive request routing). **401 Unauthorized:** The client must authenticate itself to get the requested response (request lacks valid authentication credentials). **403 Forbidden:** The client does not have access rights to the content; i.e., it is unauthorized, so the server is refusing to give the requested resource. Unlike 401, the client's identity is known to the server. **500 Internal Server Error:** The server has encountered a situation it doesn't know how to handle.
24. **Q:** Explain the concept of idempotency for HTTP methods. Which methods are typically idempotent?
A: An HTTP method is idempotent if making multiple identical requests has the same effect as making a single request. This means the resource's state will be the same after one or N identical requests. Typically idempotent methods: **GET, HEAD, OPTIONS, PUT, DELETE.** **POST** is generally not idempotent (e.g., multiple POSTs could create multiple resources). **PATCH** may or may not be idempotent depending on its implementation.
25. **Q:** What are the advantages of using REST APIs for building applications?
A: Advantages include: **Simplicity:** Uses standard HTTP methods and conventions. **Scalability:** Statelessness allows for easier scaling of servers. **Flexibility:** Supports various data formats (JSON, XML). **Decoupling:** Separates client and server, allowing them to evolve independently. **Cacheability:** Can leverage HTTP caching mechanisms. **Wide Adoption:** Well-understood and supported by many tools and libraries.
26. **Q:** Discuss some common ways to version REST APIs. What are the pros and cons of different approaches?
A: Common ways include: 1. **URI Path Versioning** (e.g., `/api/v1/users`): Clear, easy to implement and browse. Can lead to URI proliferation. 2. **Query Parameter Versioning** (e.g., `/api/users?version=1`): Doesn't clutter URI path. Less "discoverable" by just looking at URL. 3. **Custom Header Versioning** (e.g., `Accept-Version: v1` or `X-API-VERSION: 1`): Keeps URIs clean. Requires clients to set headers, less accessible via browser directly. 4. **Content Negotiation (Accept Header)** (e.g., `Accept: application/vnd.myapi.v1+json`): Technically pure RESTful way. Can be complex to implement and debug. Pros/Cons depend on discoverability, ease of use for clients, and impact on caching and routing. URI path versioning is often the most straightforward.
27. **Q:** How can you ensure the security of a REST API? (Mention a few techniques)
A: Techniques include: 1. **HTTPS (SSL/TLS):** Encrypt data in transit. 2. **Authentication:** Verify client identity (e.g., OAuth 2.0, JWT, API Keys). 3. **Authorization:** Ensure authenticated clients only access permitted resources (e.g., role-based access control). 4. **Input Validation:** Protect against injection attacks (SQLi, XSS if API outputs HTML). 5. **Rate Limiting:** Prevent abuse and DoS attacks. 6. **Proper HTTP Method Usage:** Use verbs correctly (e.g., don't use GET for state changes). 7. **Logging and Monitoring:** Detect and respond to security incidents.
28. **Q:** What is HATEOAS (Hypermedia as the Engine of Application State) and why is it considered a mature aspect of REST?
A: HATEOAS means that a client interacts with a network application entirely through hypermedia provided dynamically by application servers. In a REST API, this means responses include links (hypermedia controls) that tell the client what actions they can take next or where to find related resources. It's considered mature because it allows APIs to be more self-discoverable and evolve more gracefully, as clients don't hardcode URIs but rather follow links provided by the server. This decouples client and server more effectively.
29. **Q:** Compare and contrast REST with other API architectural styles like SOAP or GraphQL.
A: **REST vs SOAP:** SOAP (Simple Object Access Protocol) is a protocol with a stricter specification, often using XML and relying on a WSDL for defining service interfaces. REST is an architectural style, more flexible, often uses JSON, and leverages standard HTTP. SOAP can have built-in security and transaction features. REST is generally lighter and simpler. **REST vs**

GraphQL: GraphQL is a query language for APIs. Clients request exactly the data they need, preventing over-fetching or under-fetching common with REST. REST typically exposes multiple endpoints for different resources, while GraphQL often uses a single endpoint. GraphQL provides a schema and type system. REST relies on HTTP methods and status codes for semantics.

30. **Q:** If you are designing a REST API endpoint to update a specific user's email address, what HTTP verb would you use, what might the URL look like, and what data would you expect in the request body?

A: HTTP Verb: PUT if you are replacing the entire user resource (or a defined part of it that includes email) or if the update is idempotent. PATCH is often preferred for partial updates, like just changing the email. **URL:** /api/users/userId (e.g., /api/users/123) or if email is a sub-resource, /api/users/userId/email. For just an update, /api/users/userId is common. **Request Body (for PATCH):** JSON like { "email": "new.email@example.com" }. **Request Body (for PUT, if updating the whole user):** JSON like { "username": "existinguser", "email": "new.email@example.com", "other_fields": "..." }.

5 Unit V: Express Framework & MongoDB

5.1 Level 1 (Basic/Recall)

1. **Q:** What is Express.js?
A: Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications, particularly for building APIs.
2. **Q:** How do you install Express.js in a Node.js project?
A: You install it using npm (Node Package Manager) with the command: `npm install express`.
3. **Q:** What is an Express application object typically named?
A: It is conventionally named `app`. (e.g., `const app = express();`).
4. **Q:** What is a route in Express?
A: A route defines how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, etc.).
5. **Q:** What is middleware in Express?
A: Middleware functions are functions that have access to the request object (`req`), the response object (`res`), and the next middleware function in the application's request-response cycle. They can execute code, make changes to request/response objects, end the cycle, or call the next middleware.
6. **Q:** What is MongoDB?
A: MongoDB is a source-available cross-platform document-oriented database program, classified as a NoSQL database program, using JSON-like documents with optional schemas.
7. **Q:** Is MongoDB a SQL or NoSQL database?
A: MongoDB is a NoSQL database.
8. **Q:** What is a document in MongoDB?
A: A document in MongoDB is a data record, stored in BSON (Binary JSON) format. Documents are analogous to JSON objects and contain field-value pairs.
9. **Q:** What is a collection in MongoDB?
A: A collection in MongoDB is a group of documents. It is the equivalent of a table in a relational database but does not enforce a schema.
10. **Q:** How do you connect to a MongoDB instance from Node.js (mention a popular library)?
A: You typically use a driver library like the official `mongodb` Node.js driver or an ODM (Object Data Mapper) like `Mongoose`.

5.2 Level 2 (Understanding/Application)

11. **Q:** Why is Express.js a popular choice for building web applications with Node.js?
A: It's popular because it's lightweight, unopinionated (flexible), has a simple API, a rich ecosystem of middleware, and makes it easy to set up routing, handle requests/responses, and build robust APIs quickly.
12. **Q:** How do you define a simple GET route in Express that sends a "Hello World" response?
A: `app.get('/', (req, res) => { res.send('Hello World!'); });`
13. **Q:** Explain the purpose of the `request` (`req`) and `response` (`res`) objects in an Express route handler.
A: The `req` (request) object represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, etc. The `res` (response) object represents the HTTP response that an Express app sends when it gets an HTTP request. It's used to send data back to the client, set response headers, etc.
14. **Q:** How can you configure an Express application (e.g., set a view engine)?
A: You can use the `app.set()` method. For example, to set a view engine like Pug or EJS: `app.set('view engine', 'pug');` `app.set('views', './views');`. Other configurations might involve using `app.use()` for middleware.

15. **Q:** What does "rendering views" mean in the context of Express? Name a common template engine.
A: Rendering views means generating HTML to be sent to the client by combining a template file (which contains static HTML markup and dynamic placeholders) with data from the application. Common template engines include Pug (formerly Jade), EJS, and Handlebars.
16. **Q:** How can you handle form data submitted via a POST request in Express?
A: You need to use middleware to parse the incoming request body. For URL-encoded form data, use `express.urlencoded({ extended: true })`. For JSON data, use `express.json()`. The parsed data will then be available in `req.body`.
17. **Q:** How can you handle file uploads in an Express application? (Mention a common middleware)
A: You typically use a middleware designed for handling multipart/form-data, which is used for file uploads. A very common middleware for this is `multer`.
18. **Q:** Explain the basic structure of a MongoDB query to find documents.
A: Using the MongoDB shell or a driver, you use the `find()` method on a collection. It takes an optional query document as its first argument to specify filter criteria. Example: `db.collectionName.find({ status: "active" })`. An empty query document `{}` matches all documents.
19. **Q:** How do you insert a new document into a MongoDB collection using Node.js?
A: Using the `mongodb` driver or Mongoose: you would typically get a reference to your collection, then call a method like `insertOne()` (for a single document) or `insertMany()` (for multiple documents), passing the document(s) to be inserted. Example with `mongodb` driver: `collection.insertOne({ name: "Test User", age: 30 });`.
20. **Q:** What are some advantages of using MongoDB?
A: Advantages include: **Flexible Schema:** Documents can have different fields and structures. **Scalability:** Designed to scale out horizontally using sharding. **Rich Query Language:** Supports powerful querying and aggregation. **Performance:** Can be very fast for certain workloads, especially with embedded data. **Developer Friendliness:** JSON-like documents are easy to work with in JavaScript.

5.3 Level 3 (Advanced/Analytical)

21. **Q:** Discuss the middleware pipeline in Express. How does the order of middleware matter?
A: Middleware functions are executed sequentially in the order they are added to the application using `app.use()` or to a route handler. Each middleware has access to `req`, `res`, and `next()`. It can perform operations, modify `req/res`, end the request-response cycle, or pass control to the next middleware using `next()`. The order is crucial because a middleware might depend on modifications made by a previous one (e.g., a body-parser must run before a route handler tries to access `req.body`). Error-handling middleware is typically defined last.
22. **Q:** How can you implement error handling middleware in Express?
A: Error-handling middleware functions are defined with four arguments instead of three: (`err`, `req`, `res`, `next`). They must be defined after all other `app.use()` and routes calls. If any preceding middleware calls `next(err)` with an error object, Express skips subsequent non-error middleware and executes the error-handling middleware. Example: `app.use((err, req, res, next) => { console.error(err.stack); res.status(500).send('Something broke!'); });`
23. **Q:** Explain how sessions can be managed in an Express application. What are they used for?
A: Sessions are used to store user-specific data across multiple HTTP requests (since HTTP is stateless). In Express, sessions are typically managed using middleware like `express-session`. This middleware adds a session object to the request (`req.session`). It uses a cookie to send a session ID to the client, and stores session data on the server (in memory, a database like Redis, or MongoDB). Sessions are used for things like user login persistence, shopping carts, etc.
24. **Q:** Compare and contrast connecting to an SQL database versus a NoSQL database like MongoDB from an Express application. What are the conceptual differences in querying?
A: **Connection:** SQL often uses libraries like `pg` (PostgreSQL), `mysql2` (MySQL), or ORMs like Sequelize. MongoDB uses drivers like `mongodb` or ODMs like Mongoose. Connection strings

will differ in format. **Querying - SQL:** Uses Structured Query Language (SQL) with `SELECT`, `INSERT`, `UPDATE`, `DELETE` statements, often involving joins across tables, and a predefined schema. **Querying - MongoDB:** Uses a query language based on JSON-like objects. Queries are typically on a single collection. Joins are less common (data often denormalized or linked via application logic or `$lookup`). Schemas are flexible (though ODMs can enforce them). Conceptual difference: SQL is relational, focused on structured data and relationships. MongoDB is document-oriented, focused on flexible, often nested data structures.

25. **Q:** What are schemas in the context of MongoDB (often enforced by ODMs like Mongoose)? Why are they useful even though MongoDB is schema-less?
- A:** While MongoDB itself is schema-less (meaning documents in a collection don't need to have the same structure), ODMs like Mongoose allow you to define a schema at the application level. A schema defines the structure of documents, default values, data types, validators, virtuals, middleware, etc. They are useful because they provide: data validation, type casting, a clear structure for developers, default values, and can make code more maintainable and robust by ensuring data consistency within the application logic, even if the database itself doesn't enforce it.
26. **Q:** How does indexing work in MongoDB, and why is it important for performance?
- A:** Indexes in MongoDB store a small portion of the collection's data set in an easy-to-traverse form, ordered by the values of the indexed fields. When a query targets an indexed field, MongoDB can use the index to locate the matching documents quickly without scanning every document in the collection. This dramatically improves query performance, especially for large collections. Without appropriate indexes, queries can become very slow.
27. **Q:** Describe a scenario where you might use MongoDB's aggregation framework.
- A:** The aggregation framework is used for processing data records and returning computed results. It processes data through a pipeline of stages. Scenario: Calculating the average order value for different product categories in an e-commerce application. You might have stages to: 1. `$match`: Filter orders within a specific date range. 2. `$unwind`: Deconstruct an array of line items in each order into separate documents. 3. `$group`: Group documents by product category and calculate the sum of sales and count of orders per category. 4. `$project`: Reshape the output to show category, total sales, and average order value.
28. **Q:** What are some security considerations when connecting an Express app to a MongoDB database?
- A:** 1. **Authentication/Authorization:** Enable authentication on MongoDB and use strong credentials. Implement role-based access control. 2. **Network Exposure:** Bind MongoDB to localhost or a private network; avoid exposing it directly to the internet if possible. Use firewalls. 3. **Principle of Least Privilege:** The application's database user should only have the permissions necessary for its operations. 4. **Injection Attacks:** While NoSQL injection is different from SQL injection, sanitize and validate all user inputs used in queries, especially when constructing query objects dynamically. Use ODMs like Mongoose which can help. 5. **Secure Connection String:** Store connection strings with credentials securely (e.g., using environment variables, secret management services) and not hardcoded. 6. **Regular Updates:** Keep MongoDB and driver versions updated.
29. **Q:** How would you structure a larger Express application for better maintainability (e.g., separating routes, controllers, models)?
- A:** A common approach is a variation of the Model-View-Controller (MVC) pattern or a similar separation of concerns: **Routes:** Define API endpoints and map them to controller functions. Often in a dedicated `routes/` directory, with separate files per resource (e.g., `userRoutes.js`, `productRoutes.js`). **Controllers:** Contain the business logic for handling requests. They interact with services or models and send responses. Often in a `controllers/` directory. **Models (or Services/Data Access Layer):** Define data schemas (if using an ODM like Mongoose) and interact with the database. Often in a `models/` or `services/` directory. **Middleware:** Custom middleware functions can be in a `middleware/` directory. **Config:** Configuration files in a `config/` directory. The main application file (`app.js` or `server.js`) would then import and use these components.
30. **Q:** If you have an Express API endpoint that needs to fetch data from MongoDB and then make a call to an external API before sending a response, how would you manage these asynchronous operations?

A: You would use Promises or `async/await` to manage the sequence of asynchronous operations.
Using `async/await`:

```
app.get('/my-endpoint', async (req, res) => {
  try {
    const mongoData = await db.collection('myCollection').findOne({ /* query */ });
    // Process mongoData if needed
    const externalApiResponse = await fetch(`external-api-url?param=${mongoData.someField}`);
    const externalData = await externalApiResponse.json();
    // Process externalData and combine with mongoData
    res.json({ combinedData: ... });
  } catch (error) {
    console.error(error);
    res.status(500).send('Error processing request');
  }
});
```

This ensures that the MongoDB query completes first, then its result is used (if needed) for the external API call, and finally the response is sent after the external call completes.

6 Unit VI: Introduction to ReactJS

6.1 Level 1 (Basic/Recall)

1. **Q:** What is ReactJS?
A: ReactJS is an open-source JavaScript library for building user interfaces or UI components.
2. **Q:** Who developed React?
A: React was developed by Facebook (now Meta).
3. **Q:** What is the Virtual DOM?
A: The Virtual DOM (VDOM) is a programming concept where an ideal, or "virtual", representation of a UI is kept in memory and synced with the "real" DOM by a library such as ReactDOM. This process is called reconciliation.
4. **Q:** Is React a framework or a library?
A: React is generally considered a library. It focuses on the view layer and often needs to be combined with other libraries for routing, state management, etc., to form a complete framework-like solution.
5. **Q:** What is JSX?
A: JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like structures in your JavaScript code. It's compiled by tools like Babel into regular JavaScript function calls (`React.createElement()`).
6. **Q:** What are props in React?
A: Props (short for "properties") are read-only arguments passed into React components. They are used to pass data from a parent component to a child component.
7. **Q:** What is state in a React component?
A: State is an object that represents the parts of a component that can change over time, typically due to user interaction or network responses. When state changes, the component re-renders.
8. **Q:** Name one React hook.
A: Examples include `useState`, `useEffect`, `useContext`, `useReducer`. (Accept any one valid hook).
9. **Q:** How do you handle events in React?
A: Event handling in React is similar to handling events on DOM elements, but with some syntactic differences: React events are named using camelCase (e.g., `onClick` instead of `onclick`), and you pass a function as the event handler rather than a string.
10. **Q:** What is React Router used for?
A: React Router is a standard library for routing in React. It enables navigation among views of various components in a React Application, allows changing the browser URL, and keeps the UI in sync with the URL.

6.2 Level 2 (Understanding/Application)

11. **Q:** Explain the motivation for using a library like React for front-end development.
A: Motivations include: building complex, interactive UIs efficiently through a component-based architecture; improved performance via the Virtual DOM; easier state management within components; strong community support and a large ecosystem of tools and libraries; and promoting reusable UI code.
12. **Q:** How does the Virtual DOM help in improving performance?
A: Instead of directly manipulating the browser's DOM for every change (which is slow), React updates a Virtual DOM in memory. It then compares this VDOM with a snapshot of the previous VDOM (diffing) and calculates the minimal set of changes needed to update the real DOM. This batching and minimizing of direct DOM manipulations leads to better performance.

13. **Q:** Explain the concept of one-way data binding in React.

A: One-way data binding means that data flows in a single direction. In React, this is typically from parent components to child components via props. If a child component needs to modify data that originated from a parent, it usually does so by invoking a callback function passed down from the parent as a prop. This makes data flow predictable and easier to debug.

14. **Q:** What is the difference between a class component and a functional component in React (especially before hooks)?

A: Before hooks: **Class Components:** Extended from `React.Component`, could have state and lifecycle methods (e.g., `componentDidMount`). They used the `render()` method to return JSX. **Functional Components:** Simpler functions that received props and returned JSX. They were primarily used for presentational purposes as they couldn't have their own state or lifecycle methods. With hooks, functional components can now manage state, lifecycle, and other React features.

15. **Q:** How do you pass data from a parent component to a child component in React?

A: Data is passed from a parent component to a child component using props. The parent component includes attributes (props) when rendering the child component, and the child component receives these props as an argument (in functional components) or via `this.props` (in class components). Example (Parent): `<ChildComponent name="Alice" />` Example (Child Functional):

```
function ChildComponent(props) { return <h1>Hello, {props.name}</h1>; }
```

16. **Q:** How do you manage and update state within a functional component using the `useState` hook?

A: The `useState` hook is called with an initial state value. It returns an array containing two elements: the current state value and a function to update that state. Example:

```
import React, { useState } from 'react'; function Counter() { const [count, setCount] = useState(0); return (<button onClick={() => setCount(count + 1)}>Count: {count}</button>); }
```

17. **Q:** How do you use the `useEffect` hook to perform side effects, like fetching data?

A: The `useEffect` hook accepts a function that contains the side effect code. This function runs after every render by default. It can be configured to run only when certain values (dependencies) change by passing an array of dependencies as a second argument. For data fetching, it's common to run it once after the initial render by passing an empty dependency array `[]`. Example:

```
useEffect(() => { fetch('/api/data').then(res => res.json()).then(setData); }, []);
```

18. **Q:** How can you fetch data from an API using the `fetch` API in a React component?

A: Typically done within a `useEffect` hook. You use the browser's `fetch` API to make the request. The response is a Promise, so you use `.then()` to parse the JSON and then update the component's state with the fetched data.

```
useEffect(() => {
  fetch('https://api.example.com/data')
    .then(response => response.json())
    .then(data => setDataState(data)) // setDataState is from useState
    .catch(error => console.error('Error fetching data:', error));
}, []); // Empty array means run once on mount
```

19. **Q:** How do you handle basic form input and submission in React?

A: 1. **Controlled Components:** Input field values are controlled by React state. An `onChange` handler updates the state for each input. 2. A form element has an `onSubmit` handler. 3. When submitted, the handler function (e.g., `handleSubmit`) prevents the default form submission, accesses the state values, and processes them (e.g., sends to an API). Example input: `<input type="text" value={name} onChange={e} => setName(e.target.value) />`

20. **Q:** What is the purpose of `key` props when rendering lists of elements in React?

A: Keys help React identify which items in a list have changed, are added, or are removed. They should be stable, unique strings or numbers given to elements inside an array to give the elements a stable identity. This helps React efficiently update the UI by minimizing DOM manipulations when the list changes.

6.3 Level 3 (Advanced/Analytical)

21. **Q:** Discuss the key differentiators of React compared to other front-end libraries or frameworks (e.g., Angular, Vue.js).

A: React vs Angular: React is a library focused on the UI, often requiring other libraries for a full solution (routing, state management). Angular is a comprehensive framework with built-in solutions. React uses a Virtual DOM and JSX; Angular uses a real DOM (with its own change detection) and TypeScript primarily. React's learning curve for the core library can be gentler.

React vs Vue.js: Vue is also a progressive framework, often seen as easier to integrate into existing projects than React initially. Vue's core is also a view library but has a more batteries-included ecosystem that's officially maintained. Vue uses templates similar to HTML, while React uses JSX. Both use a Virtual DOM. Vue is often considered to have a gentler learning curve than Angular and perhaps even React for some. React's key differentiators: Strong focus on declarative UI, component-based architecture, Virtual DOM for performance, one-way data flow, and a vast ecosystem (though this also means more choices to make).

22. **Q:** Explain the component lifecycle in React (for class components, or how `useEffect` emulates it).

A: Class Components Lifecycle: 1. **Mounting:** `constructor()`, `static getDerivedStateFromProps()`, `render()`, `componentDidMount()` (for side effects like API calls). 2. **Updating:** `static getDerivedStateFromProps()`, `shouldComponentUpdate()`, `render()`, `getSnapshotBeforeUpdate()`, `componentDidUpdate()` (for side effects after update). 3. **Unmounting:** `componentWillUnmount()` (for cleanup). **Functional Components with Hooks:** `useEffect` emulates these: - `componentDidMount:` `useEffect(() => {...}, [])` (empty dependency array). - `componentDidUpdate:` `useEffect(() => {...}, [dep1, dep2])` (runs when dependencies change). The function can also return a cleanup function. - `componentWillUnmount:` The cleanup function returned from `useEffect(() => { return () => { /* cleanup */ }; }, [])`. State initialization is done with `useState`.

23. **Q:** What are controlled components vs. uncontrolled components in React forms?

A: Controlled Components: Form data is handled by React state. The value of form elements (like `<input>`) is driven by the component's state, and updates to the value are managed via event handlers that update the state (e.g., using `useState`). This gives React full control over the form data. **Uncontrolled Components:** Form data is handled by the DOM itself (like traditional HTML forms). You use a `ref` to get form values from the DOM when needed (e.g., on submission). They are generally simpler for basic forms but offer less control.

24. **Q:** How can you share state between components that are not directly parent-child (e.g., siblings or deeply nested components)? (Mention Context API or state management libraries)

A: 1. Lifting State Up: Move the shared state to the nearest common ancestor component and pass it down via props (and callbacks to update it). This can lead to "prop drilling" for deeply nested components. 2. **React Context API:** Provides a way to pass data through the component tree without having to pass props down manually at every level. You create a Context, provide a value at a higher level, and consume it at any lower level. Good for global state like themes, user authentication. 3. **State Management Libraries:** For more complex applications, libraries like Redux, Zustand, or Recoil provide more robust solutions for managing global application state, with features like predictable state updates, middleware, and dev tools.

25. **Q:** What are higher-order components (HOCs) and custom hooks? How do they promote code reusability?

A: Higher-Order Components (HOCs): A HOC is a function that takes a component and returns a new component, usually by wrapping the original component to add extra props or behavior. They are a pattern for reusing component logic (e.g., `withRouter` in older React Router). **Custom Hooks:** A custom hook is a JavaScript function whose name starts with "use" and that can call other hooks. They allow you to extract component logic (like stateful logic, side effects) into reusable functions. This is the modern preferred way to share logic between functional components. Both promote reusability by allowing you to abstract and share common functionality across multiple components without repeating code. Custom hooks are generally considered more flexible and easier to compose than HOCs.

26. **Q:** Explain how routing works with React Router, including dynamic routing.

A: React Router enables navigation by synchronizing the UI with the URL. 1. **Core Components:** `<BrowserRouter>` (or `<HashRouter>`) wraps the application. `<Routes>` wraps multiple `<Route>` definitions. `<Route>` maps a URL path to a React component. `<Link>` or `<NavLink>` are used for declarative navigation. 2. **Matching:** When the URL changes, React Router matches the path against the defined routes and renders the corresponding component. 3. **Dynamic Routing:** Achieved using route parameters. Example: `<Route path="/users/:userId" element={<UserProfile /> />`. The `:userId` part is a parameter. The `UserProfile` component can access this parameter (e.g., using the `useParams` hook) to fetch and display data for that specific user.

27. **Q:** What are some common strategies for form validation in React applications?

A: 1. **Manual Validation:** Implement validation logic within component state and event handlers (e.g., on blur or on change, or on submit). Check input values against rules and update state to show error messages. 2. **Using Form Libraries:** Libraries like Formik, React Hook Form, or Final Form abstract away much of the boilerplate for form handling, including validation, submission, and error handling. They often support schema-based validation (e.g., with Yup or Zod). 3. **Schema-based Validation:** Define a validation schema (e.g., using Yup or Zod) that specifies rules for each field. The form library or custom logic then uses this schema to validate the form data. 4. **Built-in HTML5 Validation:** Leverage native HTML5 validation attributes (e.g., `required`, `type="email"`, `pattern`), though often supplemented with JavaScript validation for better UX and more complex rules.

28. **Q:** Discuss the process of building and deploying a React application. What tools are typically involved?

A: Development: Use a tool like Create React App (CRA), Vite, or Next.js to set up the development environment, which includes a dev server, hot reloading, and build tools. **Building:** Run a build script (e.g., `npm run build` or `yarn build`). This uses tools like Webpack (often under the hood of CRA/Next.js) or Rollup (used by Vite for production builds) along with Babel to: - Transpile JSX and modern JavaScript to browser-compatible code. - Bundle JavaScript modules into fewer files. - Minify code (HTML, CSS, JS) to reduce size. - Optimize assets (images, fonts). - Generate static HTML, CSS, and JS files in a build folder (e.g., `build` or `dist`). **Deployment:** - The static files from the build folder are deployed to a web server or hosting platform. - Options include: Static hosting services (Netlify, Vercel, GitHub Pages, AWS S3), traditional web servers (Nginx, Apache), or PaaS (Heroku, AWS Elastic Beanstalk). - For server-side rendering (SSR) or full-stack React frameworks (like Next.js), deployment involves running a Node.js server.

29. **Q:** How does React handle conditional rendering of components or elements?

A: React handles conditional rendering in several ways, leveraging standard JavaScript: 1. **if statements:** Use standard `if` to decide which component or elements to return from a render function or a component. 2. **Logical && operator (Short-circuiting):** For simple conditions where you render something or nothing. Example: `{isLoggedIn && <UserProfile />}`. 3. **Ternary operator (condition ? exprIfTrue : exprIfFalse):** For choosing between two alternatives. Example: `{isLoggedIn ? <LogoutButton /> : <LoginButton />}`. 4. **Variables to store elements:** You can use variables to store JSX elements and then include them in the output based on conditions. 5. **Switch statements (less common directly in JSX):** Can be used in the logic part of the component.

30. **Q:** What are some performance optimization techniques you can use in a React application?

A: 1. **Using `React.memo` or `PureComponent`:** Prevent unnecessary re-renders of components if their props haven't changed. `React.memo` is for functional components, `PureComponent` for class components (does a shallow prop/state comparison). 2. **Memoizing functions and values with `useCallback` and `useMemo`:** Prevents re-creation of functions or re-computation of expensive values on every render if their dependencies haven't changed, which can help child components that rely on referential equality for props. 3. **Keys for Lists:** Use stable and unique keys when rendering lists to help React efficiently update them. 4. **Code Splitting (Lazy Loading):** Use `React.lazy` and `Suspense` to load components only when they are needed, reducing initial bundle size. 5. **Virtualization (Windowing):** For long lists or large tables, render only the visible items in the viewport using libraries like `react-window` or `react-virtualized`. 6. **Optimizing Images and Assets:** Compress images, use appropriate formats, and lazy load offscreen images. 7. **Bundle**

Analysis: Use tools like Webpack Bundle Analyzer to inspect bundle contents and identify large dependencies. 8. **Avoiding unnecessary state updates** or overly complex state structures.